# Šepet

## Varno sporočanje s šifriranjem od konca do konca

Prijava teme projekta · Računalniške storitve v oblaku 2026/27 · UL FRI

Projektna ekipa 13 »Ključarji« · Tibor Koderman, Amar Mehić, Eva Vidic · 26. 10. 2026

## 1 Osnovni podatki

| | |
|---|---|
| **Naslov projekta** | Šepet — aplikacija za varno sporočanje s šifriranjem od konca do konca |
| **Projektna skupina** | Projektna ekipa 13 |
| **Ime skupine** | Ključarji |
| **Člani** | Tibor Koderman, Amar Mehić, Eva Vidic |
| **Organizacija** | https://github.com/sepet-chat — en javni repozitorij za vsako mikrostoritev (ime in opis repozitorija sta ime in opis mikrostoritve), ločeni repozitoriji za aplikacijo, funkcije, platformo in dokumentacijo |
| **Oblak** | Microsoft Azure (Azure for Students): Azure Kubernetes Service, Blob Storage, Functions, Key Vault |

Vsak član je lastnik vsaj dveh mikrostoritev in enega prečnega področja; lastnik skrbi za kodo, teste, Helm chart in del poročila, preostala člana pa opravita pregled kode.

| Član | Mikrostoritve | Prečna področja |
|---|---|---|
| Tibor Koderman | identity-service, keys-service | platforma (AKS, Terraform, Helm, CI/CD), Keycloak, prehod API, zasnova šifriranja |
| Amar Mehić | conversation-service, message-service, realtime-service | NATS JetStream, Event Sourcing in CQRS, ScyllaDB, gRPC, GraphQL |
| Eva Vidic | media-service (+ media-functions), notification-service | aplikacija Flutter, Azure Functions, Firebase Cloud Messaging |

## 2 Kratek opis projekta

Velik del zasebne komunikacije teče prek storitev, ki jim moramo verjeti na besedo: Telegram privzetih in skupinskih klepetov ne šifrira od konca do konca[1], ponudniki pa so pod pritiskom, naj oblastem omogočijo dostop do vsebine — od predloga evropske uredbe CSAR (t. i. »Chat Control«), ki je v prvotni različici predvidel obvezno pregledovanje zasebnih sporočil in o katerem pogajanja še tečejo[2], do britanske zahteve, zaradi katere Apple od februarja 2025 novim uporabnikom v Združenem kraljestvu ne ponuja več napredne zaščite podatkov[3]. Dostop do komunikacij pa ne ostane vedno v rokah oblasti: v napadu Salt Typhoon (2024) so napadalci v omrežjih ameriških operaterjev kopirali tudi podatke o prestrezanjih, odrejenih s sodnimi odredbami, agencija CISA pa je nato ogroženim uporabnikom priporočila izključno komunikacijo, šifrirano od konca do konca[4]. **Šepet** je aplikacija za zasebno sporočanje ena na ena in v skupinah, z oblikovanim besedilom in slikami, v kateri se vsebina šifrira in dešifrira izključno na napravah udeležencev s standardiziranim protokolom MLS (RFC 9420)[5], strežnik pa hrani in posreduje samo šifrirane ovojnice in nujne metapodatke; ponudnik zato vsebine ne more prebrati, predati ali zlorabiti, tudi če bi to želel ali moral. Cilj projekta je pokazati, da je tak sistem mogoče zgraditi kot skalabilno, odporno in opazljivo domorodno oblačno aplikacijo — sedem

[1] Telegram FAQ (*cloud chats* uporabljajo šifriranje strežnik–odjemalec, *secret chats* so samo ena na ena in jih je treba ročno vklopiti): https://telegram.org/faq, https://core.telegram.org/api/end-to-end

[2] Evropska komisija, predlog uredbe COM(2022) 209: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=COM:2022:209:FIN; stališče Sveta (26. 11. 2025) brez obveznih odredb o odkrivanju: https://www.consilium.europa.eu/en/press/press-releases/2025/11/26/child-sexual-abuse-council-reaches-position-on-law-protecting-children-from-online-abuse/

[3] Apple, »Advanced Data Protection is no longer available for new users in the UK« (februar 2025): https://support.apple.com/122234

[4] FBI in CISA, skupna izjava, 13. 11. 2024: https://www.cisa.gov/news-events/news/joint-statement-fbi-and-cisa-peoples-republic-china-prc-targeting-commercial-telecommunications; CISA, *Mobile Communications Best Practice Guidance*, 18. 12. 2024: https://www.cisa.gov/sites/default/files/2024-12/guidance-mobile-communications-best-practices.pdf

[5] R. Barnes et al., *The Messaging Layer Security (MLS) Protocol*, RFC 9420, julij 2023; arhitektura: RFC 9750, april 2025.

mikrostoritev v .NET 10 s podatkovnima bazama ScyllaDB in PostgreSQL, dogodki v NATS JetStream, upravljanjem identitet v Keycloaku in odjemalcem v Flutterju, nameščenih v Azure Kubernetes Service s Helmom, CI/CD in Terraformom — ne da bi strežniški del kadarkoli potreboval ključe za dešifriranje.

**Varnostni model — kaj strežnik vidi in česa ne:**

- **Vsebina:** sporočila, slike ter imena in slike skupin se šifrirajo na napravi; priponke z naključnim ključem (AES-256-GCM), ki potuje v šifriranem sporočilu. Strežnik hrani le šifrirane ovojnice in bloke.
- **Ključi:** zasebni ključi nikoli ne zapustijo naprave; strežnik hrani javne ključe in enkratne pakete ključev (MLS KeyPackage).
- **Zaščita pred podtaknjenimi ključi:** uporabnika lahko primerjata varnostno številko (koda QR); keys-service vodi dnevnik sprememb ključev, verižen z zgoščenimi vrednostmi, aplikacija pa ob spremembi ključa stika opozori.
- **Naprejšnja tajnost in varnost po kompromitaciji:** MLS ob vsaki spremembi članstva preide v novo epoho ključev; odstranjen član novih sporočil ne more dešifrirati.
- **Metapodatki:** strežnik vidi, kdo je v katerem pogovoru (potrebno za usmerjanje), ter čas in (zapolnjeno) velikost ovojnic. Potisna obvestila ne vsebujejo vsebine, dnevniki in metrike ne vsebujejo vsebine ali seznamov stikov.
- **Zunaj obsega:** zaščita pred kompromitirano napravo in popolno skrivanje metapodatkov (npr. *sealed sender*), ki je možna nadgradnja.

# 3 Ogrodje in razvojno okolje

| Področje | Izbira | Zakaj |
|---|---|---|
| Uporabniški vmesnik | **Flutter** (Dart) za Android in splet; Riverpod, go_router, dio | ena koda za mobilno in spletno aplikacijo; spletna različica je javno dostopna za demonstracijo |
| Šifriranje E2E | **MLS (RFC 9420)** prek knjižnice **OpenMLS** (Rust, MIT) in flutter_rust_bridge, za vmesnikom `CryptoEngine`; lokalna zbirka SQLite + SQLCipher | standard IETF za skupine (1:1 je skupina dveh), naprava = član skupine; zaledje je od protokola neodvisno, zato je rezerva Signal Protocol (libsignal_protocol_dart) brez sprememb storitev |
| Mikrostoritve | **.NET 10 (LTS)**, ASP.NET Core Minimal API, Clean Architecture | zrelo ogrodje z vgrajenim OpenAPI 3.1, gRPC, preverjanjem zdravja in OpenTelemetry; en jezik za vse storitve olajša skupno predlogo |
| Knjižnice zaledja | Marten (Event Sourcing na PostgreSQL), Hot Chocolate (GraphQL), Grpc.AspNetCore, ScyllaDB C# Driver, NATS.Net, Microsoft.Extensions.Http.Resilience (Polly v8), EF Core + Npgsql | vsaka pokriva eno zahtevo predmeta brez lastnih rešitev |
| Podatkovne baze | **ScyllaDB** (sporočila), **PostgreSQL** (vse ostale storitve), Azure Blob Storage (šifrirane priponke) | ScyllaDB za časovno urejen dnevnik sporočil z veliko zapisovanja in vgrajenim TTL — Discord na njej hrani bilijone sporočil[6]; PostgreSQL za transakcijske podatke in shrambo dogodkov |
| Sporočila med storitvami | **NATS JetStream** (+ NATS KV za prisotnost) | lahek posrednik s trajnimi tokovi, ponovnim branjem in dostavo vsaj enkrat |
| Identiteta | **Keycloak** (OIDC, OAuth 2.0, organizacije, MFA) | upravljanje identitet je generična poddomena — ne gradimo je sami, temveč ga povežemo z identity-service |
| Platforma | Docker, Kubernetes (AKS 1.36), Helm, **Envoy Gateway** (Gateway API), cert-manager, KEDA, CloudNativePG, Scylla Operator | prehod z JWT (JWKS), omejevanjem hitrosti in TLS; Gateway API nadomešča upokojeni ingress-nginx |
| CI/CD in IaC | GitHub Actions (ponovno uporabni poteki, OIDC v Azure), GHCR, CodeQL, Trivy, Syft; **Terraform** (azurerm) | ista slika v vseh okoljih, samodejna namestitev v *staging*, *prod* z odobritvijo |

[6] Discord, *How Discord Stores Trillions of Messages*, 2023: https://discord.com/blog/how-discord-stores-trillions-of-messages

| Področje | Izbira | Zakaj |
|---|---|---|
| Opazovanje | OpenTelemetry, Prometheus, Grafana, Fluent Bit + Loki, Alertmanager; k6 za obremenitvene teste | metrike RED/USE, centralizirani dnevniki in opozorila na vzorce v dnevnikih |
| Brezstrežniško | **Azure Functions** (.NET 10 isolated, Flex Consumption), Event Grid, Application Insights | dogodkovno preverjanje in čiščenje priponk, ki se samodejno skalira |
| Zunanji API | **Firebase Cloud Messaging HTTP v1** (OAuth 2.0 z računom storitve) | potisna obvestila brez vsebine |
| Pogodbe | OpenAPI 3.1 (+ Scalar), AsyncAPI 3.0 + CloudEvents 1.0, Protobuf, GraphQL SDL, napake po RFC 9457 | pogodba je v repozitoriju storitve, preden napišemo kodo |
| Razvojno okolje | JetBrains Rider ali VS Code (C#), Android Studio ali VS Code (Flutter), Docker Compose in kind za lokalno gručo, Git (veji `main` in `dev`, zahtevki za združitev s pregledom) | lokalno okolje z enim ukazom; v oblak gre samo preverjena koda |

## 4 Shema arhitekture

Slika 1 prikazuje vsebnike (raven 2 modela C4) in protokole med njimi. Komunikacijo izbiramo po pravilu: če klicatelj potrebuje rezultat, da lahko nadaljuje, kliče sinhrono (REST ali gRPC, s časovno omejitvijo in nadomestnim odzivom); če gre za dejstvo, ki zanima druge storitve, objavi dogodek v NATS JetStream.

| Interakcija | Protokol | Zakaj tako |
|---|---|---|
| aplikacija → storitve (ukazi, poizvedbe) | REST prek prehoda API | javni API z OpenAPI; `Idempotency-Key` pri pošiljanju, napake RFC 9457 |
| aplikacija → seznam pogovorov | GraphQL (conversation-service) | en klic za gnezdeno sliko: člani, zadnja aktivnost, neprebrano |
| strežnik → aplikacija (nova sporočila, tipkanje, prisotnost) | WebSocket (WSS) | potiskanje z nizko zakasnitvijo, brez anketiranja |
| message-service → conversation-service (članstvo pri pošiljanju) | gRPC | klicatelj potrebuje odgovor; nizka zakasnitev, pogodba Protobuf; prekinjevalec toka in predpomnilnik |
| conversation-service → identity-service (razrešitev uporabnikov in naprav) | gRPC | preverjanje, da je novi član v isti organizaciji |
| dejstva (sporočilo shranjeno, član odstranjen, naprava preklicana …) | NATS JetStream, CloudEvents | več porabnikov, šibka sklopljenost, ponovno branje; idempotentni porabniki |
| identity-service → Keycloak | Admin REST (OAuth 2.0 *client credentials*) | uporabniki, organizacije in vloge |
| notification-service → FCM | HTTPS + OAuth 2.0 | zunanji API za potisna obvestila |
| aplikacija ↔ Blob Storage; Blob → Function → media-service | HTTPS z URL SAS; Event Grid; HTTPS (OAuth 2.0) | veliki šifrirani bloki ne obremenjujejo gruče (vzorec *valet key*) |

| Dogodek (subjekt NATS) | Proizvajalec | Porabniki | Namen |
|---|---|---|---|
| `identity.device.registered` | identity | keys, notification, realtime | lokalna kopija naprav |
| `identity.device.revoked` | identity | keys, notification, realtime | izbris ključev in žetonov, prekinitev povezav |
| `identity.organization.created` | identity | conversation, media, notification | priprava najemnika (kvote, pravila hrambe) |
| `keys.identity-key.changed` | keys | realtime | opozorilo stikom o spremenjeni varnostni številki |

| Dogodek (subjekt NATS) | Proizvajalec | Porabniki | Namen |
|---|---|---|---|
| `keys.key-packages.low` | keys | notification | tiho obvestilo napravi, naj naloži nove pakete ključev |
| `conversations.member-added` | conversation | message, media, realtime | lokalne kopije članstva za avtorizacijo in usmerjanje |
| `conversations.member-removed` | conversation | message, media, realtime | odvzem dostopa do pogovora in priponk |
| `messages.stored` | message | realtime, notification | dostava v živo ali potisno obvestilo |
| `media.attachment.available` | media | realtime | obvestilo napravam pošiljatelja |
| `media.attachment.expired` | media | message, realtime | čiščenje sklicev na priponko |

Vsak dogodek je CloudEvent s tipom, ki vsebuje različico (npr. `si.sepet.messages.stored.v1`), z `id` za idempotentne porabnike in `correlationid` za sledenje; kanali so opisani v AsyncAPI 3.0.

# 5 Mikrostoritve in njihove funkcionalnosti

Vse storitve so napisane v .NET 10 po načelih Clean Architecture (sloji *Domain*, *Application*, *Infrastructure*, *Api*), imajo končne točke `/health/live`, `/health/ready` in `/health/startup`, izvažajo metrike in dnevnike prek OpenTelemetry ter se namestijo s svojim Helm chartom.

### identity-service — računi, organizacije in naprave

*Lastništvo: Tibor Koderman · PostgreSQL `identity`.* Upravlja račune v aplikaciji in organizacije (najemnike); poverilnice, MFA in izdajo žetonov prepusti Keycloaku.

- Vključitev po prvi prijavi: povezava računa Keycloak (`sub`) z računom aplikacije, uporabniško in prikazno ime.
- Organizacije: ustvarjanje, povabila, vloge (`member`, `tenant-admin`) in pravila (hramba, največja velikost priponke); usklajeno s Keycloak Organizations prek Admin REST API.
- Naprave: registracija, pregled in preklic izgubljene naprave.
- Imenik uporabnikov znotraj organizacije (iskanje po uporabniškem imenu).
- Vmesniki: REST `/v1/me`, `/v1/users`, `/v1/devices`, `/v1/organizations/{id}/…`; gRPC strežnik `UserDirectory`; dogodki `identity.*`.

### keys-service — imenik javnih ključev

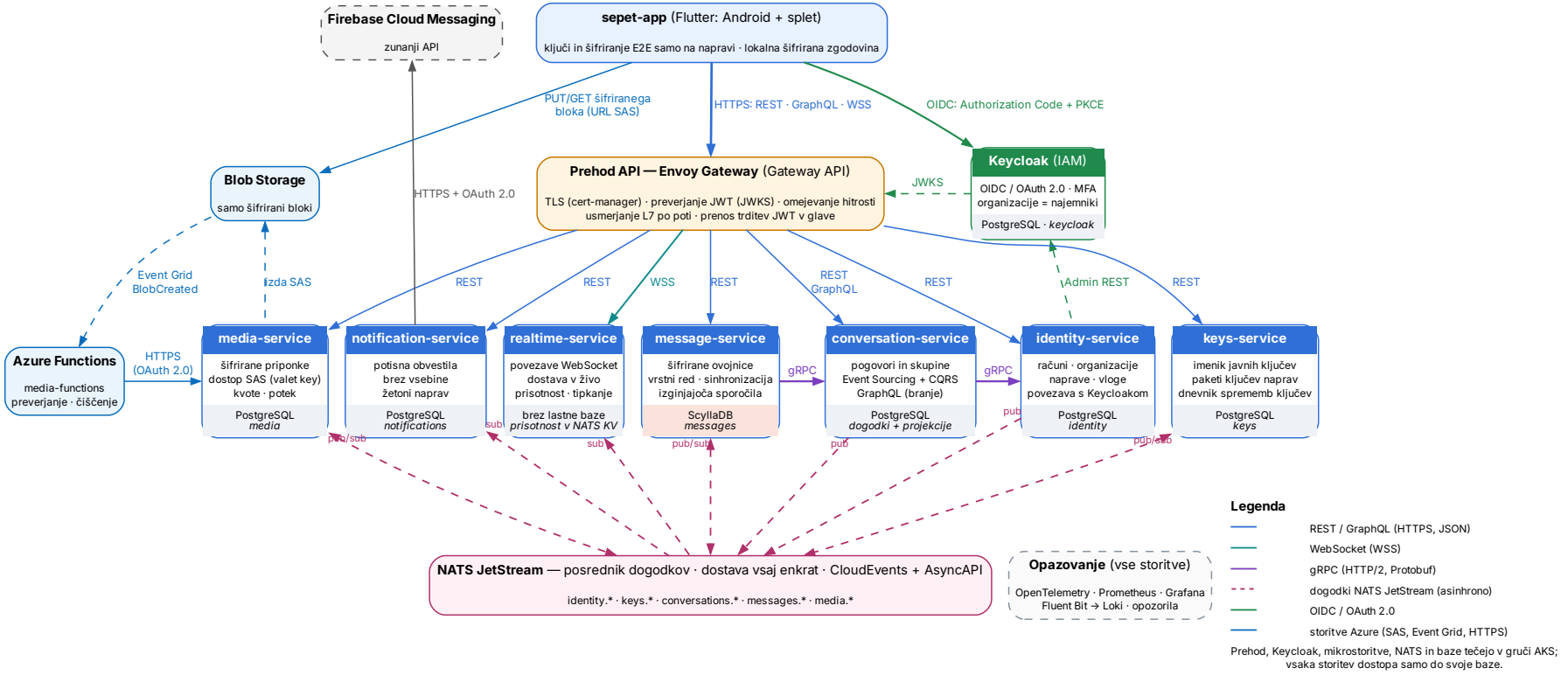
*Lastništvo: Tibor Koderman · PostgreSQL `keys`.* Skupaj z identity-service izvaja vlogo storitve za avtentikacijo iz arhitekture MLS (RFC 9750).

- Objava identitetnega ključa naprave in zaloge paketov ključev (KeyPackage), s preverjanjem podpisa ob nalaganju.
- Atomarni prevzem paketa ključev za vsako napravo novega člana — vsak paket se porabi natanko enkrat (`FOR UPDATE SKIP LOCKED`).
- Dnevnik sprememb ključev, verižen z zgoščenimi vrednostmi, za varnostne številke in opozorila ob spremembi ključa.
- Opozorilo, ko napravi zmanjkuje paketov ključev; ob preklicu naprave izbris njenih ključev.
- Vmesniki: REST `/v1/devices/{id}/key-packages`, `/v1/key-packages/claim`, `/v1/users/{id}/identity-keys`, `/v1/key-log`; dogodki `keys.*`; porablja `identity.device.*`.

### conversation-service — pogovori in skupine (Event Sourcing + CQRS)

*Lastništvo: Amar Mehić · PostgreSQL `conversations` (Marten).* Upravlja pogovore ena na ena in skupine; stanje je zaporedje dogodkov.

- Pisalni model (agregat `Conversation`): ustvari pogovor ali skupino, dodaj ali odstrani člana, spremeni vlogo, nastavi čas izginjanja sporočil, posodobi šifrirane nastavitve (ime in slika skupine sta šifrirana).
- Shramba dogodkov: `ConversationCreated`, `MemberAdded`, `MemberRemoved`, `RoleChanged`, `SettingsChanged` …; integracijski dogodki se objavijo iz nje, zato ni problema dvojnega zapisa.
- Bralni modeli (projekcije): seznam pogovorov uporabnika z zadnjo aktivnostjo in članstvo;




**Slika 1:** Začetna shema arhitekture aplikacije Šepet s komunikacijskimi protokoli. Vsaka mikrostoritev ima lastno podatkovno bazo (spodnji pas okvirja); do tujih podatkov dostopa le prek API-jev ali dogodkov.

GraphQL (Hot Chocolate) za aplikacijo, gRPC strežnik `Membership` za druge storitve.
- Ponovno predvajanje dogodkov obnovi projekcije iz nič (demonstracija zahteve 12).
- Vmesniki: REST `/v1/conversations`, `/v1/conversations/{id}/members`; GraphQL `/graphql`; gRPC `Membership`; dogodki `conversations.*`.

**message-service — dostava šifriranih sporočil**

*Lastništvo: Amar Mehić · ScyllaDB `messages`.* Dostavna storitev v smislu arhitekture MLS: hrani in ureja šifrirane ovojnice, ne da bi jih lahko prebrala.

- Sprejem šifrirane ovojnice (REST z `Idempotency-Key`), preverjanje članstva pošiljatelja prek gRPC in dodelitev zaporedne številke v pogovoru.
- Urejanje sprememb skupine (MLS Commit): za vsako epoho sprejme natanko eno spremembo s pogojnim zapisom (*lightweight transaction*), zato vsi člani uporabijo spremembe v istem vrstnem redu.
- Hramba: particija (pogovor, časovno vedro), urejeno po zaporedni številki; TTL za izginjajoča sporočila.
- Sinhronizacija naprave po vrnitvi na splet (`GET …/messages?after={seq}`) in kazalci prebranega po napravi.
- Vmesniki: REST `/v1/conversations/{id}/messages`, `/v1/conversations/{id}/commits`; gRPC odjemalec `Membership`; dogodek `messages.stored`; porablja `conversations.member-*`.

**realtime-service — dostava v živo**

*Lastništvo: Amar Mehić · brez lastne baze (prisotnost v NATS KV s časom poteka).* Skalira se po številu povezav, ločeno od message-service.

- Povezave WebSocket, overjene z žetonom JWT; dostava novih ovojnic, sprememb pogovorov in opozoril o ključih povezanim napravam.
- Prisotnost in indikator tipkanja (efemerno).
- Porablja `messages.stored`, `conversations.member-*`, `keys.identity-key.changed` in `identity.device.revoked` (prekine povezave preklicane naprave).

**media-service — šifrirane priponke**

*Lastništvo: Eva Vidic · PostgreSQL `media` + Azure Blob Storage.*

- Izda kratkotrajen URL SAS za nalaganje ali prenos (vzorec *valet key*); bloki potujejo neposredno v Blob Storage, mimo gruče.
- Metapodatki priponke (velikost, zgoščena vrednost šifriranega bloka, stanje, čas poteka) in kvote po organizaciji.
- Prenos dovoli le članom pogovora (lokalna kopija članstva iz dogodkov).
- Vmesniki: REST `/v1/attachments`, `/v1/attachments/{id}`; notranji webhook za Azure Function; dogodki `media.*`.

**media-functions — brezstrežniški funkciji (Azure Functions)**

*Lastništvo: Eva Vidic · .NET 10 isolated, Flex Consumption, Application Insights.*

- `AttachmentValidator` (Event Grid `BlobCreated`): preveri velikost in da blok ni nešifrirana slika (prepoznavni začetni bajti formatov JPEG, PNG …), rezultat sporoči media-service (OAuth 2.0 *client credentials*).
- `AttachmentJanitor` (časovni sprožilec): briše potekle bloke in sporoči izbrisane priponke.

**notification-service — potisna obvestila**

*Lastništvo: Eva Vidic · PostgreSQL `notifications`.*

- Registracija žetonov FCM po napravah in nastavitve obveščanja (utišani pogovori).
- Ob `messages.stored` napravam brez povezave pošlje potisno obvestilo **brez vsebine** prek FCM HTTP v1 (OAuth 2.0 z računom storitve); aplikacija se zbudi in sama prenese šifrirane ovojnice.
- Ponovni poskusi, prekinjevalec toka in omejevanje hitrosti proti FCM; neveljavne žetone odstrani.

**sepet-app — uporabniški vmesnik (Flutter)**

*Lastništvo: Eva Vidic; zaslone za svoje storitve prispeva vsak član.*

- Podstrani: prijava, seznam pogovorov, pogovor (oblikovano besedilo, slike), nova skupina in člani,

profil z napravami in varnostno številko (koda QR), nastavitve, skrbništvo organizacije (samo `tenant-admin`).
- Prijava z OIDC (*Authorization Code* + PKCE), komunikacija z REST, GraphQL in WebSocket; ključi v varni shrambi naprave (Android Keystore), zgodovina v šifrirani lokalni zbirki.
- Oblikovano besedilo kot omejen Markdown (krepko, ležeče, prečrtano, koda, seznami, citati, povezave, omembe), ki ga upodobi odjemalec; strežnik ga vidi le šifriranega.

# 6 Primeri uporabe

| Oznaka | Primer uporabe | Raven | Sodelujoče storitve |
|---|---|---|---|
| PU-01 | Registracija in prijava (OIDC + PKCE), registracija naprave in objava javnih ključev | osnovni | Keycloak, identity, keys |
| PU-02 | Iskanje uporabnika v organizaciji in začetek pogovora ena na ena | osnovni | identity, keys, conversation, message |
| PU-03 | Pošiljanje in prejemanje šifriranega sporočila — v živo in po ponovni povezavi | osnovni | message, conversation, realtime |
| PU-04 | Pisanje oblikovanega besedila (krepko, ležeče, koda, seznami, povezave, omembe) | osnovni | aplikacija, message |
| PU-05 | Seznam pogovorov z zadnjo aktivnostjo in številom neprebranih (GraphQL) | osnovni | conversation |
| PU-06 | Potisno obvestilo brez vsebine za napravo brez povezave | osnovni | message, notification, FCM |
| **PU-07** | **Pošiljanje šifrirane slike z opisom v skupinski pogovor (kompleksni primer)** | napredni | media, Functions, conversation, message, realtime, notification |
| PU-08 | Ustvarjanje skupine ter dodajanje in odstranjevanje članov (nova epoha ključev) | napredni | conversation, identity, keys, message, realtime |
| PU-09 | Preverjanje varnostne številke s kodo QR in opozorilo ob spremembi ključa stika | napredni | keys, realtime |
| PU-10 | Izginjajoča sporočila in priponke s časom poteka | napredni | conversation, message, media, Functions |
| PU-11 | Skrbništvo organizacije: povabila, dodelitev vloge `tenant-admin`, pravila hrambe | napredni | identity, Keycloak |
| PU-12 | Pregled in preklic izgubljene naprave (izbris ključev, žetonov in povezav) | napredni | identity, keys, notification, realtime |
| PU-13 | Obnova bralnih modelov s ponovnim predvajanjem dogodkov (operater) | napredni | conversation |
| PU-14 | Delovanje ob izpadu posamezne storitve (npr. media ali notification) | napredni | vse |

**PU-07 — pošiljanje šifrirane slike z opisom v skupinski pogovor.** *Akterji:* Ana (pošiljateljica), Bor (povezan), Cene (brez povezave), vsi člani skupine v organizaciji FRI. *Predpogoj:* Ana je prijavljena in ima veljaven žeton; skupina je v epohi MLS *n*.

1. **Aplikacija (Ana):** Ana izbere sliko in doda opis. Aplikacija odstrani metapodatke EXIF, izdela sličico, ustvari naključni ključ *K* in sliko šifrira (AES-256-GCM); izračuna SHA-256 šifriranega bloka.
2. **Prehod API → media-service (REST):** prehod preveri JWT in omejitev hitrosti; media-service preveri kvoto organizacije in Anino članstvo, shrani metapodatke (stanje `pending`) in vrne URL SAS za nalaganje (samo zapis, 10 minut, omejena velikost).
3. **Aplikacija → Blob Storage:** šifrirani blok naloži neposredno, mimo gruče.
4. **Blob Storage → Event Grid → Azure Function:** `AttachmentValidator` preveri velikost in da blok ni nešifrirana slika, nato pokliče media-service (OAuth 2.0); ta priponko označi kot `available` in objavi `media.attachment.available`.
5. **Aplikacija → message-service (REST):** sestavi sporočilo {id priponke, *K*, SHA-256, tip, mere, opis v Markdownu}, ga šifrira s skupinskim ključem epohe *n* in pošlje z `Idempotency-Key`.
6. **message-service → conversation-service (gRPC):** preveri, ali je Anina naprava član skupine v epohi *n* (časovna omejitev, ponovni poskus, prekinjevalec toka; ob izpadu kratkotrajen predpomnilnik članstva).

7. **message-service → ScyllaDB, NATS:** shrani ovojnico z zaporedno številko (in TTL, če skupina uporablja izginjajoča sporočila) ter objavi `messages.stored`.
8. **realtime-service → Bor (WebSocket):** dostavi ovojnico; Borova aplikacija sporočilo dešifrira, prek media-service dobi URL SAS za branje, prenese blok, preveri SHA-256, ga dešifrira s *K* in prikaže.
9. **notification-service → FCM → Cene:** Cene ni povezan (prisotnost v NATS KV), zato dobi potisno obvestilo brez vsebine; aplikacija se zbudi, prenese manjkajoče ovojnice (`?after={seq}`) in prikaže lokalno obvestilo.

*Odpornost:* če notification-service ne deluje, se sporočilo vseeno shrani in dostavi povezanim napravam, obvestila pa se pošljejo, ko se storitev vrne (trajni porabnik JetStream). Če ne deluje media-service, besedilna sporočila delujejo naprej, nalaganje slike pa se ponovi z eksponentnim odlogom. Če ne deluje FCM, se v notification-service odpre prekinjevalec toka.

# 7 Pokritost zahtev predmeta

Ciljamo na vseh 20 osnovnih zahtev (100 točk) in vse bonus zahteve; bonus točke so rezerva za morebitne izgube drugje.

| # | Zahteva | Točke | Načrtovana izvedba |
|---|---|---|---|
| 1 | Repozitorij | 2 | organizacija GitHub, javni repozitorij na mikrostoritev, README, veji `main` in `dev`, zaščitena `main` |
| 2 | Mikrostoritve, cloud-native | 5 | 7 mikrostoritev + GUI, Clean Architecture, 12-faktorska načela, manifesti Kubernetes prek Helma |
| 3 | Baza na storitev | 3 | 5 × PostgreSQL, ScyllaDB, ločene poverilnice, migracije in začetni podatki |
| 4 | Dokumentacija | 3 | repozitorij `docs`: model C4, zapisi ADR, navodila za razvijalce, tehnično poročilo |
| 5 | Dokumentacija API | 4 | OpenAPI 3.1 (Scalar) s primeri in napakami RFC 9457; AsyncAPI, `.proto`, GraphQL SDL |
| 6 | CI/CD | 6 | GitHub Actions: testi → slika v GHCR → samodejno v *staging* → *prod* z odobritvijo |
| 7 | Helm | 5 | chart za vsako storitev, `values-dev`, `values-staging`, `values-prod` |
| 8 | Namestitev v oblak | 6 | Azure AKS, javni dostop prek prehoda z domeno in TLS |
| 9 | Zunanji API | 4 | FCM HTTP v1 z OAuth 2.0 v notification-service |
| 10 | Večnajemniškost | 6 | organizacije v Keycloaku, trditev o najemniku v JWT; RLS v PostgreSQL in najemnik v ključu particije ScyllaDB; ločen imenski prostor in baze za izoliranega najemnika |
| 11 | Preverjanje zdravja | 5 | `/health/live`, `/health/ready`, `/health/startup` + sonde Kubernetes |
| 12 | Event Sourcing in CQRS | 6 | conversation-service: shramba dogodkov, projekcije, bralni model GraphQL, ponovno predvajanje |
| 13 | Izolacija in odpornost | 6 | časovne omejitve, ponovni poskusi, prekinjevalec toka, pregrade; demonstracija izpadov |
| 14 | Upravljanje konfiguracije | 5 | datoteke → okoljske spremenljivke → ConfigMap z osvežitvijo brez ponovne namestitve → Secrets in Key Vault → Azure App Configuration (zastavice funkcionalnosti) |
| 15 | Grafični vmesnik | 5 | Flutter (Android + splet), več kot dve podstrani, REST |
| 16 | Brezstrežniška funkcija | 6 | Azure Functions: Event Grid `BlobCreated` in časovni sprožilec, Application Insights, skaliranje Flex Consumption |
| 17 | GraphQL in gRPC | 5 | GraphQL v conversation-service; gRPC message → conversation in conversation → identity |
| 18 | Pretakanje dogodkov | 6 | NATS JetStream; 6 storitev objavlja ali porablja dogodke |
| 19 | Centralizirani dnevniki | 6 | Fluent Bit → Loki → Grafana; opozorila na vzorce (npr. niz neuspelih prijav, napake podpisov) |
| 20 | Metrike | 6 | OpenTelemetry → Prometheus → Grafana; RED/USE; obremenitveni testi k6 in optimizacija |
| 21 | Vmesna predstavitev (bonus) | +10 | 14.–18. 12. 2026 |

| # | Zahteva | Točke | Načrtovana izvedba |
|---|---|---|---|
| 22 | Terraform (bonus) | +3 | moduli za AKS, Storage, Functions, Key Vault in opazovanje; `plan` in `apply` v CI |
| 23 | Prehod API (bonus) | +8 | Envoy Gateway: usmerjanje L7, TLS, JWT, omejevanje hitrosti, prenos trditev v glave |
| 24 | IAM, OAuth2, OIDC (bonus) | +4 | Keycloak: OIDC + PKCE, vloge (`tenant-admin`), zaščita API-jev z OAuth 2.0 |

# 8 Časovni načrt in tveganja

| Obdobje | Mejnik |
|---|---|
| do 26. 10. | prijava teme, organizacija in repozitoriji, ključne odločitve |
| 27. 10.–8. 11. | »hodeče okostje«: predloga storitve, CI/CD, Helm, AKS s Terraformom, Keycloak, prijava v aplikaciji; preizkus integracije MLS v Flutter |
| 9.–22. 11. | jedro: ključi in šifriranje 1:1, message-service z ScyllaDB, realtime-service, NATS (1. kolokvij 17. 11.) |
| 23. 11.–13. 12. | skupine (Event Sourcing, CQRS, GraphQL, gRPC), slike in Azure Functions, obvestila FCM, prehod API, opazovanje |
| 14.–18. 12. | vmesna predstavitev |
| 19. 12.–3. 1. | večnajemniškost, odpornost, upravljanje konfiguracije, metrike in optimizacija |
| 4.–11. 1. 2027 | tehnično poročilo in priprava predstavitve; oddaja 11. 1. 2027 |

- **Integracija MLS v Flutter (zlasti splet):** časovno omejen preizkus v prvem tednu; ker zaledje hrani le neprozorne pakete ključev in ovojnice, lahko brez sprememb storitev preklopimo na Signal Protocol.
- **Proračun Azure (100 USD na študenta):** gručo ustavimo, ko je ne uporabljamo, razvijamo lokalno (Docker Compose, kind), uporabljamo majhna vozlišča in brezplačne ali najcenejše stopnje storitev.
- **Obseg (7 storitev):** skupna predloga storitve in ponovno uporabni poteki CI/CD; vsak član je lastnik svojih storitev.